

Plan de Desarrollo por Fases: IDE Pseudocódigo

Hoja de ruta iterativa estructurada en 4 fases para la construcción, integración de seguridad y despliegue de la PWA de pseudocódigo.

1. RESUMEN EJECUTIVO DE LA HOJA DE RUTA

El desarrollo de la PWA se abordará mediante un enfoque progresivo y modular. Para minimizar riesgos técnicos, la arquitectura se construirá desde el núcleo de compilación (Engine) hasta la integración de la interfaz, los bloqueos de seguridad y el soporte offline de PWA.

Fase	Enfoque Principal	Entregable Clave	Complejidad
Fase 1	Core Engine & AST	Intérprete funcional en Web Worker aislado con timeout guard.	Alta
Fase 2	UI & Security Guards	Editor interactivo con bloqueo de portapapeles y sesión efímera.	Media
Fase 3	Módulo DFD	Lienzo independiente de diagramas de flujo (React Flow) sin autoconversión.	Media-Alta
Fase 4	Exportación PDF Protegido	Modal interactivo con huella digital esteganográfica y Hash criptográfico anti-copia.	Media-Baja
Fase 5	PWA & Polish	Service Workers, soporte offline y empaquetado PWA final.	Baja

2. DESGLOSE DETALLADO DE FASES DE DESARROLLO

Fase 1: Motor del Intérprete y Compilador Client-Side

Sprints 1 - 2

Objetivo: Construir la gramática del lenguaje, el analizador sintáctico y el entorno de ejecución asíncrono aislado sin depender de una interfaz gráfica compleja.

- Análisis Léxico y Sintáctico:** Definición de reglas gramaticales en español (`Si`, `Mientras`, `Para`, etc.) utilizando Chevrotain o Nearley.js.
- Generador AST:** Construcción del Árbol de Sintaxis Abstracta con validación de errores sintácticos en tiempo real.
- Tabla de Símbolos & Entorno:** Mapeo de variables con tipado estático obligatorio (`entero`, `decimal`, `cadena`, `booleano`).
- Aislamiento Web Worker:** Ejecución del motor dentro de un Worker secundario comunicación por `postMessage`.
- Execution Loop Guard:** Hilo de monitoreo que aborta la ejecución (.terminate()) si sobrepasa un tiempo límite de 30000ms.

Fase 2: Interfaz de Usuario y Controles de Seguridad

Sprints 3 - 4

Objetivo: Integrar el editor visual CodeMirror 6 / Monaco, construir la consola I/O e implementar las restricciones de copia e integridad.

- Editor de Código Personalizado:** Coloreado de sintaxis (Syntax Highlighting) y resaltado visual de líneas con errores.
- Clipboard Guard (Anti-Copia):** Intercepción y anulación de eventos `paste`, `drop`, `dragover` y menú contextual.
- Session Lifecycle Guard:** Limpieza automática de la memoria RAM del editor al detectar eventos `visibilitychange` o `blur`.
- Consola Virtual Asíncrona:** UI interactiva para desplegar salidas (`Mostrar`) y solicitar entradas (`Leer`) pausando la ejecución.

Fase 3: Módulo Independiente de Diagramas de Flujo (DFD)

Sprint 5

- Objetivo:** Integrar un lienzo gráfico vectorial (React Flow) independiente para el diseño manual de diagramas ANSI sin conversión automática con el código.
- **Lienzo Interactivo:** Editor visual con zoom, pan y símbolos ANSI (Terminal, Proceso, Decisión, I/O).
 - **Independencia Estricta:** Conmutador en UI (``Pseudocódigo`` vs ``DFD``) garantizando desarrollo autónomo de ambas lógicas.
 - **Aislamiento de Sesión:** Limpieza efímera del canvas ante eventos de desenfoque (``blur`/`visibilitychange``).

Fase 4: Pop-up de Exportación y Sistema de Identificación Única Anti-Copia

Sprint 6

- Objetivo:** Implementar el modal de recolección de metadatos académicos y el motor de marcas de agua esteganográficas e identificadores únicos criptográficos para impedir la suplantación y el intercambio de archivos entre estudiantes.
- **Modal Interactivo Académico:** Captura de Nombre (forzado a MAYÚSCULAS), Materia, Curso (``D1``, ``N1``, etc.), Nombre del Trabajo y selección de contenido (``Pseudocódigo``, ``DFD`` o ``Ambos``).
 - **Huella Digital Única (SHA-256 Digest):** Generación de firma criptográfica inviolable basada en la combinación de metadatos + timestamp exacto + hash del contenido.
 - **Marcas de Agua Forenses (Anti-Suplantación):** Microestampe esteganográfico invisible y patrón de fondo visible con el Nombre y Curso del estudiante incrustado en cada página.
 - **Inmovilización del Documento:** Encriptación con permisos de solo lectura y aplanado vectorial de estructuras para evitar edición directa.

Fase 5: Empaquetado PWA, Modo Offline y Pruebas

Sprint 7

- Objetivo:** Convertir la aplicación web en una PWA instalable con alta resiliencia y funcionamiento offline completo.
- **Web App Manifest:** Configuración de metadatos, íconos adaptativos y modo de despliegue ``standalone``.
 - **Estrategia de Service Worker:** Implementación de almacenamiento en caché `*Cache-First*` para assets estáticos e intérprete.
 - **Pruebas de Estrés y Cross-Browser:** Validación en Chrome, Safari, Firefox y Edge sobre dispositivos móviles y laptops.

Fase 4: Empaquetado PWA, Modo Offline y Pruebas

Sprint 6

- Objetivo:** Convertir la aplicación web en una PWA instalable con alta resiliencia y funcionamiento offline completo.
- **Web App Manifest:** Configuración de metadatos, íconos adaptativos y modo de despliegue ``standalone``.
 - **Estrategia de Service Worker:** Implementación de almacenamiento en caché `*Cache-First*` para assets estáticos e intérprete.
 - **Pruebas de Estrés y Cross-Browser:** Validación en Chrome, Safari, Firefox y Edge sobre dispositivos móviles y laptops.
 - **Optimización de BUNDLE:** Ajustes de minificación para asegurar una carga inicial menor a $\$1.5\backslash\text{text{ segundos}}\$$.

3. MATRIZ DE RIESGOS E MITIGACIÓN

Riesgo Técnico	Impacto	Estrategia de Mitigación
Bucle Infinito en Intérprete	Alto	El hilo principal destruye el Web Worker vía <code>`Worker.terminate()`</code> a los 3 segundos de inactividad.